

MAXSQUASH

PLAYER CLUB MEMBERSHIP APIs

Version 1.3

Prepared by Usama

For Backend Team

PLAYER SIDE

View joined clubs, add another club, and remove an existing club after login.

CLUB SIDE

Registration-added members must be approved immediately and include membership numbers.

1. Objective

This document defines the Version 1.3 backend scope for player club membership management and the required member payload changes in club registration. The implementation must follow the existing authentication and response conventions used by MaxSquash.

2. API Scope Summary

Method	Endpoint	Authentication	Purpose	Pagination
GET	get-player-clubs	Bearer token	Return clubs where the logged-in player is an approved member.	Yes
POST	add-player-club	Bearer token	Submit a request to join another registered club.	No
POST	remove-player-club	Bearer token	Remove an existing club membership with a mandatory reason.	No
POST	auth/register/club	As currently implemented	Include membership number for each supplied member and save them as approved.	No

3. Common API Standards

- All player-side endpoints require the authenticated player bearer token.
- The API must use the existing project response structure. No global response contract change is required.
- IDs must be valid existing records and must not reference soft-deleted clubs or users.
- All write operations must be transactional and must create an auditable membership history entry.
- Dates should be returned in ISO 8601 format and status values should use consistent lowercase enums.

4. Player Side - Get Player Clubs

Endpoint

GET get-player-clubs

Returns the clubs where the authenticated player currently has an approved membership. Each item must include sufficient club details for display in the mobile application.

Query Parameters

Parameter	Type	Required	Description
page	integer	No	Page number. Default: 1.

per_page	integer	No	Records per page. Default: 15; maximum: 50.
search	string	No	Search by club name or membership number.

Required Club Data

- club_id
- club_name
- logo / image
- address or location
- contact details where available
- membership_number
- membership_status
- joined_at

Example Response

```
{
  "status": 200,
  "message": "Player clubs retrieved successfully",
  "data": [
    {
      "club_id": 21,
      "club_name": "City Squash Club",
      "club_logo": "https://example.com/clubs/21.png",
      "location": "Riyadh",
      "membership_number": "CSC-1045",
      "membership_status": "approved",
      "joined_at": "2026-07-20T10:30:00Z"
    }
  ],
  "pagination": {
    "current_page": 1,
    "per_page": 15,
    "total": 1,
    "last_page": 1
  }
}
```

5. Player Side - Add a New Club

Endpoint

POST add-player-club

Allows an authenticated player to request membership in another registered club after login.

Request Body

```
{
  "club_id": 35,
  "membership_number": "MSC-22018"
}
```

Business Rules

- club_id and membership_number are required.
- The player cannot submit a duplicate request for a club where membership is already approved or pending.
- The selected club must be active and available in the system.
- The new membership request must be saved with status pending until the selected club accepts or rejects it.
- The selected club must receive a notification containing the player basic details and membership number.
- Approval adds the player to the club member list. Rejection must notify the player and preserve the request in history.

Example Success Response

```
{
  "status": 201,
  "message": "Club membership request submitted successfully",
  "data": {
    "club_id": 35,
    "membership_number": "MSC-22018",
    "membership_status": "pending"
  }
}
```

6. Player Side - Remove Existing Club

Endpoint

POST remove-player-club

Allows the authenticated player to remove an approved club membership. A reason is mandatory for audit and club visibility.

Request Body

```
{
  "club_id": 21,
  "reason": "I am no longer a member of this club."
}
```

Business Rules

- club_id and reason are required.
- The player may remove only their own existing approved membership.
- The reason must be trimmed and should follow the project maximum text-length standard.

- The operation should deactivate or soft-delete the membership instead of removing historical data permanently.
- The club must receive a notification that the player has left, including the supplied reason.
- The removal action must be saved in membership history with player ID, club ID, reason, and timestamp.

Example Success Response

```
{
  "status": 200,
  "message": "Club membership removed successfully",
  "data": {
    "club_id": 21,
    "membership_status": "removed",
    "reason": "I am no longer a member of this club."
  }
}
```

7. Club Side - Registration Member Changes

Existing Endpoint

POST auth/register/club

Keep the existing club registration request and response flow unchanged except for the member payload additions described below.

Required Change in members Array

```
{
  "members": [
    {
      "player_id": 101,
      "membership_number": "CSC-1045"
    },
    {
      "player_id": 205,
      "membership_number": "CSC-1188"
    }
  ]
}
```

Registration Rules

- Each member object supplied during club registration must include player_id and membership_number.
- membership_number is required for every selected member and must not be blank.
- Members added by the club during registration must be saved directly with membership status approved.
- These members do not require a separate approval request because the club is explicitly adding and verifying them during registration.
- The same player must not be duplicated within the members array.
- Membership numbers should be unique within the same club unless the existing business rules explicitly allow otherwise.
- The membership record should save approved_at and identify the approval source as club_registration.
- Existing club registration response format must remain unchanged.

8. Membership Status Behaviour

Action	Initial Status	Next Action	Notification
Player requests another club	pending	Club accepts or rejects	Notify selected club
Club adds member during registration	approved	No approval required	Optional player confirmation
Player removes approved club	removed	No further action	Notify club with reason
Club rejects player request	rejected	Player may submit a new request later if allowed	Notify player

9. Validation and Error Handling

HTTP Status	Scenario	Expected Message
400	Invalid or missing request fields	Validation error with field-level details
401	Missing or invalid player token	Unauthenticated
404	Club or membership does not exist	Requested resource not found
409	Duplicate approved/pending membership	Membership already exists or request already pending
422	Business rule violation	Unable to process membership request
500	Unexpected server failure	Internal server error

10. Acceptance Criteria

1. Authenticated players can retrieve a paginated list of their approved clubs with club and membership details.
2. Authenticated players can request membership in another active club by providing club ID and membership number.
3. Authenticated players can remove an approved club membership only by providing a reason.
4. Club notifications are generated for new membership requests and player-initiated removals.
5. The club registration request accepts membership_number for every member entry.
6. Members supplied during club registration are stored with approved status immediately.
7. Duplicate memberships, invalid clubs, and invalid player IDs are rejected safely.
8. All membership additions, approvals, rejections, and removals remain available in history.
9. Existing auth/register/club response structure is not changed.

Implementation Note

Endpoint names may be aligned with the project routing convention, but the defined behaviour, validation, authentication, and membership status rules must remain the same.